

Analyse du Projet Spring Boot : Amélioration d'application TODO

MELLAK Khadija

1 Introduction

Ce document présente une analyse détaillée du projet **TODO**, un backend Spring Boot permettant la gestion d'une liste de tâches. L'application suit une architecture en couches (Controller–Service–Repository) et communique avec une base de données PostgreSQL. Une interface frontend Angular consomme ensuite l'API REST exposée par le backend.

L'objectif de ce document est d'expliquer clairement les choix de conception, le rôle de chaque couche, la communication interne du projet, les configurations essentielles, et d'ajouter une proposition de valeur ajoutée que nous intégrerons ultérieurement.

2 Architecture générale du projet

Le projet backend adopte une architecture standard pour les applications Spring Boot :

- une entité représentant le modèle de données ;
- un repository gérant l'accès à la base ;
- un service implémentant la logique métier ;
- un contrôleur exposant les endpoints REST ;
- une configuration pour la sécurité CORS ;
- un gestionnaire global d'exceptions.

Cette organisation permet une séparation claire des responsabilités, un code maintenable et une facilité d'évolution.

3 Détails des composants du projet

3.1 L'entité Todo

L'entité représente une ligne dans la table `todos`. Elle contient trois attributs : un identifiant, un titre obligatoire, et un état booléen indiquant si la tâche est complétée.

```
@Entity
@Table(name = "todos")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Todo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Le titre est obligatoire")
    private String title;

    private boolean completed = false;
}
```

Spring Boot, grâce à JPA et Hibernate, génère automatiquement la table correspondante et mappe les valeurs entre la base de données et l'application.

3.2 Le repository

Le repository hérite de `JpaRepository`, ce qui offre nativement toutes les opérations CRUD (Create, Read, Update, Delete). Aucun code supplémentaire n'est nécessaire pour interagir avec PostgreSQL.

```
public interface TodoRepository extends JpaRepository<Todo, Long> {
}
```

Spring se charge de générer toutes les implémentations à partir de cette interface.

3.3 Le service TodoService

Le service centralise la logique métier. Il constitue une couche intermédiaire entre le contrôleur et la base de données. Le professeur a suivi une approche simple : chaque méthode délègue le travail au repository.

```
@Service
```

```

@RequiredArgsConstructor
public class TodoService {

    private final TodoRepository repository;

    public List<Todo> findAll() {
        return repository.findAll();
    }

    public Todo create(Todo todo) {
        return repository.save(todo);
    }

    public Todo update(Long id, Todo todo) {
        todo.setId(id);
        return repository.save(todo);
    }

    public void delete(Long id) {
        repository.deleteById(id);
    }

    public Todo findById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new RuntimeException("Todo not found: "
                + id));
    }
}

```

La méthode `findById` illustre la gestion d'erreurs en lançant une `RuntimeException` si la ressource n'existe pas.

3.4 Le contrôleur REST

Le contrôleur est le point d'entrée de l'API. Il expose quatre routes HTTP : GET, POST, PUT et DELETE.

```

@RestController
@RequestMapping("/api/todos")
@CrossOrigin(origins = "http://localhost:4200", allowCredentials = "true")
@RequiredArgsConstructor
public class TodoController {

    private final TodoService service;

```

```

@GetMapping
public List<Todo> findAll() {
    return service.findAll();
}

@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Todo create(@Valid @RequestBody Todo todo) {
    return service.create(todo);
}

@PutMapping("/{id}")
public Todo update(@PathVariable Long id, @Valid @RequestBody Todo
    todo) {
    return service.update(id, todo);
}

@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable Long id) {
    service.delete(id);
}
}

```

Le contrôleur suit une logique REST respectant les bonnes pratiques :

- GET /api/todos : récupérer toutes les tâches;
- POST /api/todos : créer une nouvelle tâche;
- PUT /api/todos/{id} : mettre à jour une tâche;
- DELETE /api/todos/{id} : supprimer une tâche.

L'annotation `@CrossOrigin` autorise Angular (port 4200) à consommer l'API.

3.5 La gestion globale des exceptions

Le professeur a ajouté un gestionnaire centralisé pour capturer les erreurs et renvoyer des réponses structurées au format `ProblemDetail`.

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ProblemDetail handleAll(Exception ex) {
        return ProblemDetail.forStatusAndDetail(

```

```

        HttpStatus.INTERNAL_SERVER_ERROR, ex.getMessage());
    }

    @ExceptionHandler(RuntimeException.class)
    public ProblemDetail handleRuntime(RuntimeException ex) {
        return ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
    }
}

```

Cette approche améliore la maintenabilité et évite des réponses non contrôlées.

3.6 La configuration CORS

Une classe `CorsConfig` est présente mais vide. Elle peut être utilisée plus tard pour configurer globalement les règles CORS.

3.7 Configuration de la base de données

Dans `application.properties`, le professeur configure PostgreSQL :

- URL de connexion ;
- identifiants ;
- dialecte Hibernate ;
- logs SQL ;
- mode de génération du schéma.

Ces paramètres permettent à Spring d'initialiser automatiquement la base et de gérer les migrations structurelles simples (`ddl-auto=update`).

4 Interaction avec le frontend Angular

Le frontend Angular envoie des requêtes HTTP vers l'API Spring Boot, principalement via un `TodoService` Angular qui appelle :

- GET `/api/todos` pour afficher la liste ;
- POST `/api/todos` pour ajouter un élément ;
- PUT `/api/todos/{id}` pour le modifier ;
- DELETE `/api/todos/{id}` pour le supprimer.

Grâce à `@CrossOrigin(...)` dans le backend, Angular est autorisé à effectuer ces requêtes depuis le port 4200.

5 I. Motivation de la Valeur Ajoutée

Le projet original TODO développé par le professeur est fonctionnel, mais il présente plusieurs limites :

- Il n’offre aucune sécurité pour l’accès aux données.
- Les données des todos sont limitées : un titre et un indicateur de complétion.
- L’expérience utilisateur est basique et ne bénéficie pas de fonctionnalités temps réel.

Nous avons donc décidé d’ajouter des améliorations majeures afin de :

- améliorer l’expérience utilisateur,
- sécuriser l’accès aux données,
- manipuler des données plus riches (priorité, dates, catégories),
- optimiser les performances et la réactivité,
- ajouter une couche d’automatisation et de notifications temps réel.

Ces ajouts constituent notre valeur ajoutée pour ce projet.

6 Ajout n°1 : Système d’authentification JWT sécurisé

6.1 Pourquoi sécuriser une API REST

Les API ouvertes exposent toutes les données à n’importe quel utilisateur. La sécurisation via JWT permet :

- de restreindre l’accès aux utilisateurs authentifiés,
- de lier les données à un utilisateur spécifique,
- de garantir la confidentialité et l’intégrité des informations.

6.2 Entité User et endpoints d’authentification

Nous avons ajouté une entité **User** avec les champs classiques : email, mot de passe, rôle, etc. Les endpoints ‘/auth/login’ et ‘/auth/register’ permettent respectivement la connexion et l’inscription.

6.3 Lien Todo → User

Chaque todo est maintenant associé à un utilisateur :

```
@ManyToOne
@JoinColumn(name = "user_id")
private User user;
```

Filtre JWT et SecurityConfig

Le filtre `JwtFilter` intercepte chaque requête pour vérifier le token JWT et authentifier l'utilisateur. La configuration Spring Security moderne (`SecurityConfig`) définit les endpoints sécurisés et les règles d'accès selon les rôles.

7 Ajout n°2 : Amélioration de la logique métier

7.1 Nouveaux champs ajoutés

L'entité `Todo` a été enrichie avec :

- `priority` : LOW, MEDIUM, HIGH
- `createdAt` : date de création automatique
- `dueDate` : date d'échéance
- `category` : WORK, STUDY, HOME, OTHER
- `description` : texte libre

7.2 Modification de l'Entity et DTO

Un `TodoDTO` a été ajouté pour exposer proprement les données au frontend. Le mapping entre Entity et DTO assure que seules les informations nécessaires sont partagées.

7.3 Nouveaux endpoints

- GET `/todos/filter?priority=HIGH` : filtre par priorité
- GET `/todos/search?q=...` : recherche texte dans titre et description

7.4 Consommation Angular

Les nouveaux endpoints sont consommés par des méthodes dans `todo.service.ts` :

```
getTodosByPriority(priority: string): Observable<Todo[]> { ... }
searchTodos(query: string): Observable<Todo[]> { ... }
```

Les composants affichent les filtres et la recherche via un `select` et un `input`.

7.5 Bilan

Les todos sont désormais plus riches et les utilisateurs peuvent filtrer et rechercher directement depuis l'interface.

8 Ajout n°3 : Notifications et WebSockets

Backend Spring Boot

- Ajout de `spring-boot-starter-websocket` dans le `pom.xml`.
- Configuration WebSocket avec STOMP et endpoint `/ws-todos`.
- Service de notification (`NotificationService`) pour diffuser les modifications de todos.
- Mise à jour des méthodes `create`, `update`, `delete` dans `TodoService` pour envoyer les notifications.

8.1 Frontend Angular

- Service `WebsocketService` utilisant STOMPJS et SockJS pour se connecter au backend.
- Composants Angular (`TodoListComponent`) abonnés aux notifications pour recharger la liste automatiquement.

8.2 Notifications d'échéance

Un `@Scheduled` task vérifie périodiquement les todos arrivés à échéance et envoie une notification temps réel.

8.3 Bilan

- Réactivité totale côté frontend
- Notifications instantanées sur création, modification, suppression ou échéance des todos

9 V. Intégration technique Frontend

9.1 Consommation de l'API sécurisée

Angular utilise un `AuthService` et des interceptors pour gérer les JWT et sécuriser les requêtes HTTP.

9.2 AuthGuard pour protéger les routes

```
@Injectable({
  providedIn: 'root'
})
export class AuthGuard implements CanActivate {
  constructor(private authService: AuthService, private router: Router)
  {}
  canActivate(): boolean {
    if (!this.authService.isLoggedIn()) {
      this.router.navigate(['/login']);
      return false;
    }
    return true;
  }
}
```

9.3 Enrichissement des composants

Les composants Angular ont été adaptés pour :

- afficher la priorité, les dates, les catégories et la description
- supporter les filtres et la recherche
- mettre à jour automatiquement grâce aux notifications WebSocket